

離散的なトークンの扱い

自然言語処理＞モデル＞離散性



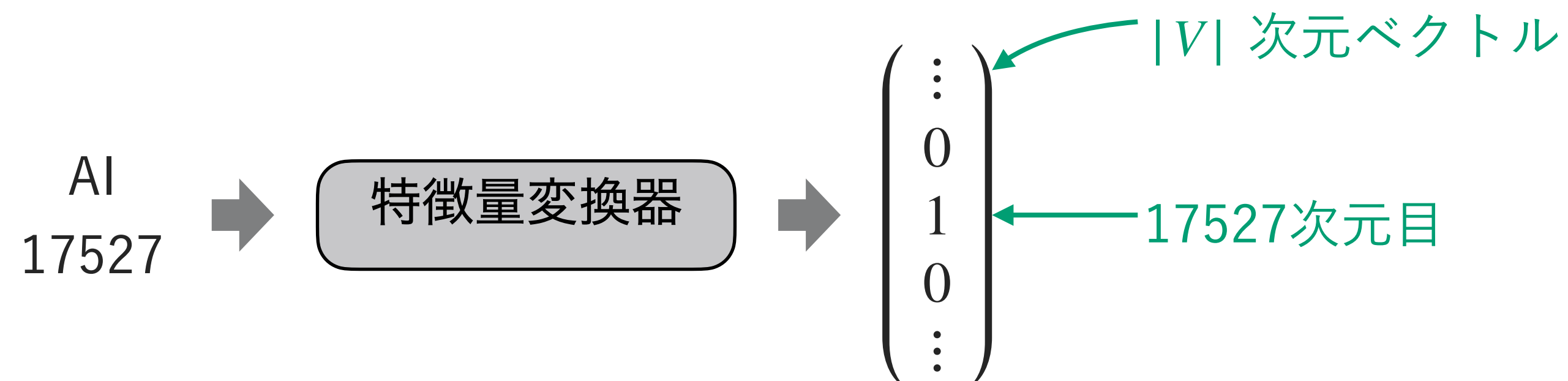
どうやって離散的なトークン（トークン番号）をモデルで扱うか？

- テキストデータを処理する上での考慮事項は主に2つある
 - **離散的**：数値ではなく「文字」という離散的なものを入出力できる必要がある
 - ▶ そのままでは，モデルの学習はおろか，四則演算などの計算すらできない
 - **可変長**：入力される文字数が予め決まっていない
 - ▶ そのまま順伝播型ニューラルネットなどで扱うことは出来なさそう
- ここでは，離散的なトークンの扱い方の話を解説

自然言語処理＞モデル＞トークンの離散性

自然言語データの処理には、トークンIDの列の入出力をするモデルが必要

- 最も単純な方式の一つは、**ワンホット (one-hot) ベクトル**を使うというもの
 - あらかじめトークンの一覧である語彙 V は決まっている
 - そこで、トークンのID番目の次元だけ1, 他は0という、語彙サイズ $|V|$ と同じ次元のベクトルをトークンの特徴量として使う



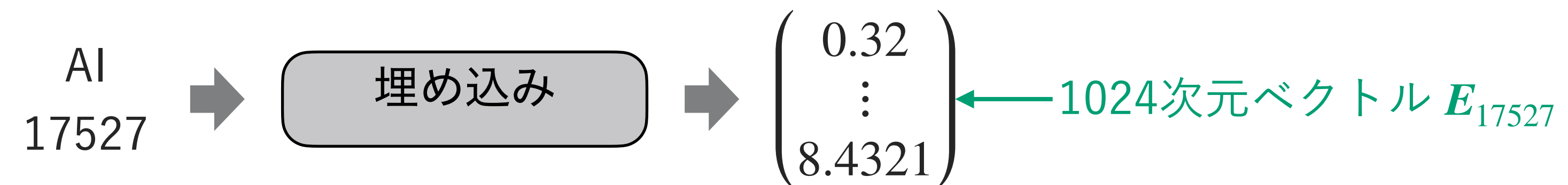
【用語】ワンホットベクトルによる特徴量のベクトルは、ほとんどの成分が0でスカスカなので「疎な特徴量 (sparse feature)」と呼ばれる。また、非ゼロ成分が局在しているということを指して局所表現 (local representation) ということもある。

自然言語処理＞モデル＞トークン埋め込み

現在の標準的な方法は、トークン埋め込みの学習という扱い方

■ トークン埋め込み (token embedding)

- トークンの1つ1つに対して、その意味をよく反映した特徴量ベクトルを用意
- 例えば、1024次元のトークン埋め込みを、トークン1つごとに用意する



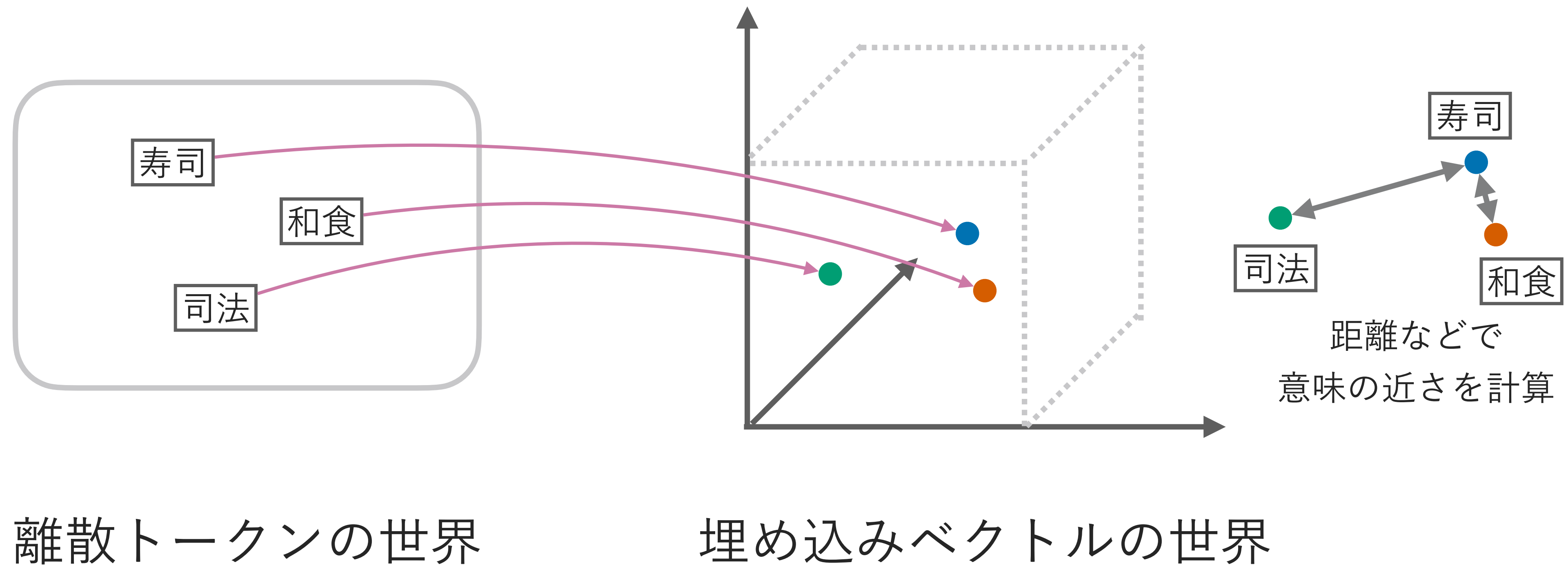
■ どんなベクトルを割り当てるのか？全てデータから学習する

- 「意味」を上手く反映した特徴量になるようなベクトルを割り当てたい

【用語】単語埋め込み (word embedding) ということも多い。分散表現 (distributed representation) と呼ばれることもある。また、疎な特徴量という言い回しとの対比として、密な特徴量 (dense feature) と呼ばれることもある。【補足】トークンを1024次元空間に点として埋め込むようなものなので、埋め込みという。

自然言語処理＞モデル＞トークン埋め込み

ベクトル空間に埋め込むことで、類似度や関連性などの計算がしやすくなる



自然言語処理＞モデル＞分布仮説

分布仮説：トークンのベクトル表現を得るための基本的なアイデア

■ 分布仮説 (distributional hypothesis)

- 単語の意味は、その周辺に出現する単語の分布で形成されるという考え方

■ 例) Q. 次の ☂ という単語の意味は何でしょう？

- 1. 「☂のお忘れ物が大変多くなっております」と車内放送が流れた.
- 2. 急に空が暗くなってきたので、彼は持っていた☂を開いた.

【補足】 答えは傘. ちなみに ☂ は「チャッ」という感じの発音で、実際にクメール語で傘という意味.

自然言語処理＞モデル＞単語埋め込みの学習



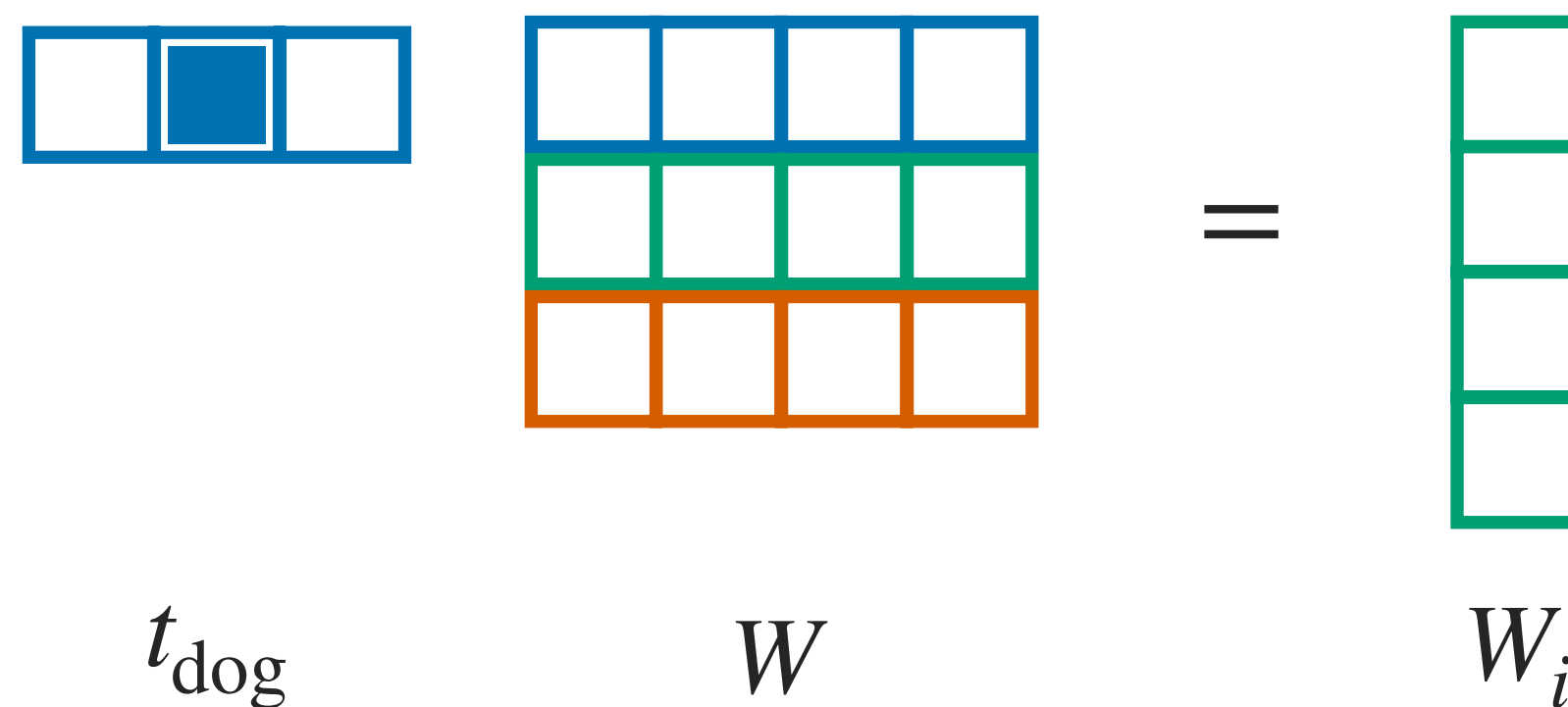
埋め込みの研究はサブワードトークナイザーが使われ始める前の時代に進展

- 2013年に発表された word2vec などが代表的な学習法
 - 主に，形態素解析器などを使用して，単語＝トークンとして扱われていた時代
- 現在では，トークン埋め込みごとモデル全体を学習することが多い
- コンセプトは現在も活きているので軽く解説します
 - CBoW（continuous bag of words）とSkip-gramがあるが，Skip-gramを解説
 - 学習の詳細は省略
- また，のちにfastTextなどの後続モデルも発展

自然言語処理 > モデル > 埋め込み層

埋め込み層 (embedding layer) はモデルの入力層に使われる

- トークンは語彙数 $|V|$ 次元のワンホットベクトル（ここでは行ベクトルとする）
 - 例) “dog” $\rightarrow t_{\text{dog}} = (\dots, 0, 1, 0, \dots)$ (“dog”のトークン番号 i 番目の成分だけ1)
- 単語埋め込みパラメーター（埋め込み次元は H ）は、 $\mathbb{R}^{V \times H}$ の行列 W で保持する
 - t_{dog} に W を右から掛けると、結果は W の i 行目を抜き出したベクトルになる。



【補足】 実際には、行列の積ではなく、トークン番号に対応するパラメーターを呼び出す対応表となっている。

自然言語処理 > モデル > トークン埋め込みの学習

Skip-gramの学習法：「文脈語として出現するか否か」で対照学習

- 訓練データの文中の単語 T_i ごとに、前後数個の単語を「文脈語」と呼ぶ

$$\mathbf{x}_i = \cdots \underbrace{T_{i-2}}_{\text{context}} \underbrace{T_{i-1}}_{\text{context}} \underbrace{T_i}_{\text{context}} \underbrace{T_{i+1}}_{\text{context}} \underbrace{T_{i+2}}_{\text{context}} \cdots$$

- 正例：文脈語 T_c とのペア (T_i, T_c) ，負例：ランダムな $T \in V$ とのペア (T_i, T)
- 次のモデルで、正例・負例の確率論的二値分類を学習（最尤法ベースの学習）

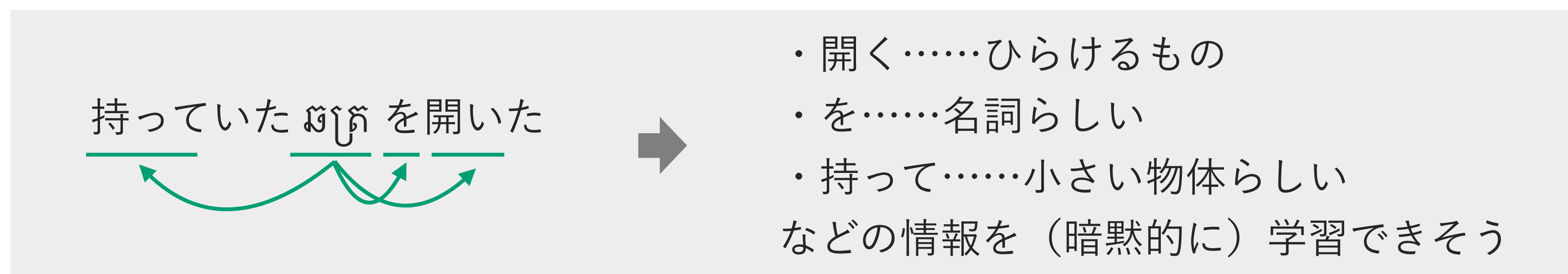


【補足】最終的に、学習後の $E_1, \dots, E_{|V|}$ が埋め込みベクトルとなり、読み取り用の $D_1, \dots, D_{|V|}$ は捨ててしまう。また、実際は最終層は正例・負例に出現したペアのみ計算すればよい。負例単語の分布は、訓練データの相対頻度を $3/4$ 乗して規格化したものが用いられる。負例は1個ではなく多数サンプリングされる。

自然言語処理＞モデル＞トークン埋め込みの学習

word2vec (Skip-gramおよびCBow) の単語埋め込みは大きな成功を収めた

- Skip-gramの学習の直観：語の意味は周囲の単語から分かる（分布仮説の実現）
 - 文脈語は近くなり，非文脈語は遠くなるように，ベクトル埋め込みを学習



- Skip-gramの学習の結果：意味論的な計算がうまくいく事例が有名
 - 例) $E_{\text{Vietnam}} + E_{\text{capital}}$ に一番近いのは E_{Hanoi} ，二番目は $E_{\text{Ho Chi Minh City}}$ など

【補足】埋め込みベクトルを使った計算が何でも狙い通りにいくわけではないが，ここに挙げた以外にも多くの計算結果があるので偶然とは言い難く，これらの結果は衝撃を与えた。【出典】Table 5 of Mikolov et al. (2013). Distributed representations of words and phrases and their compositionality. Proceedings of the 27th International Conference on Neural Information Processing Systems - Volume 2, 2, 3111–3119.

自然言語処理＞モデル＞離散性



トークンは離散的であるという問題は、埋め込みを学習することで解決

- 単語レベルでの埋め込みは、word2vecの2つのモデル（CBoWとSkip-gram）が実用的に高速な学習法を確立し普及の決定打になった。のちにfastTextなどがそれを進化させた
- 現在ではモデル全体の一部としてトークン埋め込みごと学習されるのが普通だが、トークンの埋め込みというモデリング方針は変わらず使われている
- トークン埋め込みは、言語データを現代的な方法でコンピューターで取り扱う上での基盤的技術となっている

ロードマップ

